

Contrôle de connaissances

Programmation orientée objet C++

Durée 1h – CORRECTION

Partie 1 – Tableaux de dates

1-a Classe date avec constructeur

Correction

```
1 class date
2 {
3     public :
4         int j, m, a;
5         date(int j=0, int m=0, int a=0) {
6             this->j = j;
7             this->m = m;
8             this->a = a;
9         }
10 };
```

Le constructeur utilise des valeurs par défaut (0,0,0), ce qui permet d'écrire `date d;` sans argument.

1-b Fonction de comparaison inferieure

Correction

```
1 bool date::inferieure(const date & d2) const {
2     if (this->a != d2.a) return this->a <= d2.a;
3     if (this->m != d2.m) return this->m <= d2.m;
4     return this->j <= d2.j;
5 }
```

On compare d'abord les années, puis les mois, puis les jours. Retourne `true` si `*this` est antérieure ou égale à `d2`.

1-c-a Plus petite et plus grande date d'un tableau

Correction

```
1 void min_max(date * T, int N, date & min, date & max) {
2     min = T[0];
3     max = T[0];
4     for (int i = 1; i < N; i++) {
5         if (T[i].inferieure(min)) min = T[i];
6         if (max.inferieure(T[i])) max = T[i];
7     }
8 }
```

On passe `min` et `max` par référence pour retourner deux valeurs. On initialise les deux à `T[0]` puis on parcourt le tableau.

1-c-b Complétion du main pour 1-c-a

Correction

```

1  int main() {
2      int N = 100;
3      date *T = initialiser_alea(T, N);
4
5      // A ce stade T est un tableau de N dates
6      date min, max;
7      min_max(T, N, min, max);
8
9      cout << "Plus_petite_date:"
10         << min.j << "/" << min.m << "/" << min.a << endl;
11      cout << "Plus_grande_date:"
12         << max.j << "/" << max.m << "/" << max.a << endl;
13
14      delete[] T;
15      return 0;
16  }

```

1-d-a nb_anterieures sur tableau

Correction

```

1  int nb_anterieures(date * T, int N, const date & D) {
2      int compteur = 0;
3      for (int i = 0; i < N; i++) {
4          if (T[i].inferieure(D)) compteur++;
5      }
6      return compteur;
7  }

```

On parcourt le tableau et on incrémente le compteur chaque fois qu'une date est antérieure (ou égale) à D.

1-d-b anterieures sur tableau – retourne un sous-tableau

Correction

```

1  date * anterieures(date * T, int N, const date & D, int & taille) {
2      // 1. Compter pour allouer exactement
3      taille = nb_anterieures(T, N, D);
4      if (taille == 0) return NULL;
5
6      // 2. Allouer et remplir
7      date * R = new date[taille];
8      int idx = 0;
9      for (int i = 0; i < N; i++) {
10         if (T[i].inferieure(D)) {
11             R[idx++] = T[i];
12         }
13     }
14     return R;
15 }

```

On utilise `nb_anterieures` pour connaître la taille avant d'allouer, ce qui évite une sur-allocation. La taille du nouveau tableau est retournée via le paramètre `taille` passé par référence.

1-d-c Complétion du main pour 1-d-b

Correction

```

1  int main() {
2      int N = 100;
3      date *T = initialiser_alea(T, N);
4
5      // Recuperer toutes les dates <= 01/12/2000
6      date D(1, 12, 2000);
7      int tailleR = 0;
8      date *R = anterieures(T, N, D, tailleR);
9
10     // Afficher le tableau R
11     if (R == NULL) {
12         cout << "Aucune date anterieure a 01/12/2000." << endl;
13     } else {
14         for (int i = 0; i < tailleR; i++) {
15             cout << R[i].j << "/" << R[i].m << "/" << R[i].a <<
16                 endl;
17         }
18         delete[] R;
19     }
20     delete[] T;
21     return 0;
22 }
```

Partie 2 – Liste chaînée de dates

Les structures données :

```

1  class Maillon {
2      friend class liste_dates ;
3      date D ;
4      Maillon *suivant ;
5  public :
6      Maillon(int j=1, int m=1, int a=2000) {
7          (*this).D.j=j ; (*this).D.m=m ; (*this).D.a=a ;
8      }
9  };
10
11  class liste_dates {
12      Maillon *tete ;
13  public :
14      liste_dates() { (*this).tete = NULL ; }
15      ...
16  };

```

2-b insérer_tete**Correction**

```
1 void liste_dates::insérer_tete(int j, int m, int a) {
2     Maillon * nouveau = new Maillon(j, m, a);
3     nouveau->suivant = tete;
4     tete = nouveau;
5 }
```

On alloue un nouveau maillon, on le fait pointer vers l'ancienne tête, puis on met à jour tete.

2-c-a nb_dates_antérieures sur liste**Correction**

```
1 int liste_dates::nb_dates_antérieures(const date & D) const {
2     int compteur = 0;
3     Maillon * p = tete;
4     while (p != NULL) {
5         if (p->D.inferieure(D)) compteur++;
6         p = p->suivant;
7     }
8     return compteur;
9 }
```

On parcourt la liste maillon par maillon et on compte les dates antérieures à D.

2-c-b antérieures sur liste – retourne un tableau**Correction**

```
1 date * liste_dates::antérieures(const date & D, int & taille) const
2 {
3     // 1. Compter d'abord (pour allouer exactement)
4     taille = nb_dates_antérieures(D);
5     if (taille == 0) return NULL;
6
7     // 2. Allouer et remplir
8     date * R = new date[taille];
9     int idx = 0;
10    Maillon * p = tete;
11    while (p != NULL) {
12        if (p->D.inferieure(D)) {
13            R[idx++] = p->D;
14        }
15        p = p->suivant;
16    }
17    return R;
18 }
```

Même stratégie que pour le tableau : on compte d'abord via `nb_dates_antérieures` pour allouer exactement la bonne taille.

Partie 3 – Arbre binaire de dates

Les structures données :

```

1 class Noeud {
2     friend class arbre_dates ;
3     date D ;
4     Noeud *fg ;    // fils gauche
5     Noeud *fd ;    // fils droit
6     public :
7         ...
8 };
9
10 class arbre_dates {
11     Noeud *racine ;
12     public :
13     arbre_dates() { racine = NULL ; }
14     ...
15 };

```

3-c-a nb_dates_antérieures récursif sur arbre

Correction

```

1 // Fonction auxiliaire recursive (privee ou libre)
2 int nb_dates_antérieures_rec(Noeud * n, const date & D) {
3     if (n == NULL) return 0;
4     int compteur = 0;
5     if (n->D.inferieure(D)) compteur = 1;
6     compteur += nb_dates_antérieures_rec(n->fg, D);
7     compteur += nb_dates_antérieures_rec(n->fd, D);
8     return compteur;
9 }
10
11 // Methode publique de arbre_dates
12 int arbre_dates::nb_dates_antérieures(const date & D) const {
13     return nb_dates_antérieures_rec(racine, D);
14 }

```

On explore récursivement le sous-arbre gauche et le sous-arbre droit. Le cas de base est un nud NULL qui contribue 0.

3-c-b antérieures sur arbre – retourne un tableau

Correction

```

1 // Auxiliaire pour remplir le tableau (parcours recursif)
2 void remplir_rec(Noeud * n, const date & D, date * R, int & idx) {
3     if (n == NULL) return;
4     if (n->D.inferieure(D)) R[idx++] = n->D;
5     remplir_rec(n->fg, D, R, idx);
6     remplir_rec(n->fd, D, R, idx);
7 }
8
9 date * arbre_dates::antérieures(const date & D, int & taille) const
10 {

```

```
10 // 1. Compter pour allouer exactement
11 taille = nb_dates_anterieures(D);
12 if (taille == 0) return NULL;
13
14 // 2. Allouer et remplir
15 date * R = new date[taille];
16 int idx = 0;
17 remplir_rec(racine, D, R, idx);
18 return R;
19 }
```

On réutilise `nb_dates_anterieures` (question 3-c-a) pour connaître la taille avant d'allouer. Le remplissage est fait par un second parcours récursif.